

LiveTape.ai

William Huynh

Live: <https://app.livetape.ai> (marketing site at <https://livetape.ai>)

Code sample: <https://gist.github.com/williammh/a1ccdc038f186247220c070aee1ed974>

Built and run solo: backend, frontend, and infrastructure. Live in production right now, trading real market data across crypto, equities, and prediction markets.

What it does

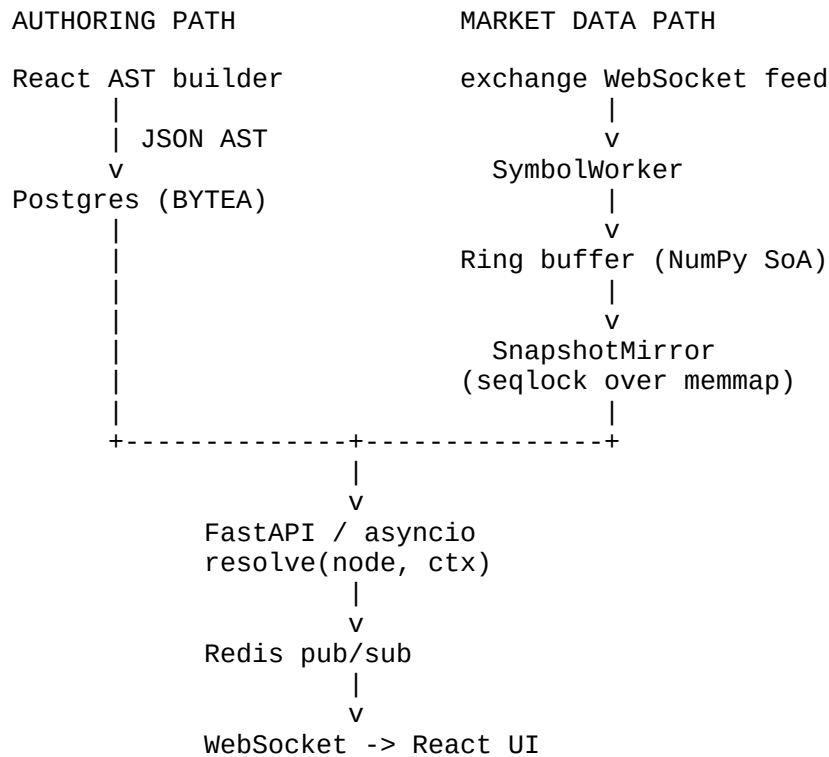
A trading strategy gets built visually in the browser: select indicators, set a threshold, wire up conditions. The backend then runs it against live market data, continuously, for as long as the strategy is turned on.

Coming up with a trading idea is easy now; any LLM can generate one. Actually running it, correctly, live, against a real feed, without falling over, is a different problem, and that's where most of the engineering effort went: getting market data in fast and reliably, keeping time semantics sane when the network jitters, coordinating between processes without locking anything up, running many strategies at once on one machine, and storing/versioning strategies in a format that won't break as the schema evolves.

Every tick gets evaluated, not just the close of each bar the way most no-code trading tools do. Internal latency is under 50ms: tick in, through the ring buffer, through a lock-free snapshot, through the AST, decision out, no database call anywhere in that path. What happens after that (how fast the broker fills the order, any signal provider upstream) is downstream of the engine and not counted here.

How it's wired together

Two things happen mostly independently and meet at evaluation. On one side, a user edits a strategy in the browser and it gets stored. On the other, market data streams in continuously and gets kept warm in shared memory. Once a bar closes, the engine pulls both together and runs the strategy.



The left side only runs once, when an agent starts up. The right side never stops, and it crosses over into the evaluation process at SnapshotMirror without ever taking a lock.

A few decisions worth a closer look

The market-data handoff is lock-free. Data comes in over WebSocket and gets bucketed into a fixed-size NumPy ring buffer: plain arrays, $O(1)$ push, nothing ever shifts. The strategy-evaluation process reads that buffer through a memory-mapped file, guarded by a seqlock rather than a real lock, a queue, or a database round-trip. The writer is on the critical path of a live feed and can't afford to ever wait on a slow reader, and readers only care about the latest state anyway, not every update in between. There's no shared-memory object and nothing brokering it; the file on disk is the boundary. The protocol, and the one platform assumption it leans on, are documented in the code sample above.

The frontend never runs any trading logic. A condition built in the visual editor ($SMA(50)[1] > SMA(200)[1]$) gets turned into a JSON tree (an ExpressionNode), and that's all the frontend ever does with it: build the tree, render the tree. The backend's `resolve(node, ctx)` walks that exact same tree shape recursively. Both sides check the tree against the same schema (Zod on one side, Pydantic on the other), so there's one grammar and the two halves can't drift apart from each other.

Bar references are relative, not absolute. An indicator doesn't point at a fixed slot in an array. It's `parent_offset + node.bar_index`, and that adds up as the tree nests. So `SMA(50)[1]` inside a bigger expression means "one bar back from wherever the parent

landed," and the same piece of logic can be reused at a different offset without touching it. This one rule is what the whole resolver hangs off of.

The clock is deterministic, never `time.time()`. Every evaluation runs against `ctx.current_time_s`, an internal clock that only moves forward and only moves in response to real events (a bar closing, a tick, or, if the market goes quiet, a synthetic heartbeat). That's what lets backtests and live runs share the exact same code path: in a backtest the clock is just set straight from the replayed timestamps instead. Nothing in the resolver ever calls the system clock, which is the only reason a backtest is reproducible at all.

No Python loop ever touches the full buffer. There's a rule on this project: nothing may loop in plain Python over a full ring-buffer-sized array (172,800 elements) on the event loop. A loop that size would block long enough to time out any HTTP request coming in at the same moment. So every bulk operation on market data is vectorized NumPy: fancy indexing, `reduceat`, slice assignment. Python glues things together; the array math runs in C.

A strategy exists in three formats on purpose. It's authored as a plain dict over the API, stored as UTF-8 JSON bytes in Postgres, and converted once at agent start into a `np.uint8` array, the form the engine actually runs on. Three small, tested functions sit at that boundary (`rules_to_bytes`, `rules_from_bytes`, `rules_to_uint8`), so the storage format, the wire format, and the in-memory format can each change on their own without a migration touching the other two.

One event loop runs up to 50 strategies at once. Each one gets its own runtime state and never hits the database on a per-cycle basis. State loads once when the agent starts and just lives in memory after that. Logs get written out asynchronously, off to the side, so they don't slow anything down.

Stack

Backend: Python 3.13, FastAPI, `asyncpg`, `SQLModel`, Redis, NumPy. Frontend: TypeScript, React, Zustand, Zod, `lightweight-charts`. Docker Compose for dev/preview/prod, running on OCI ARM64, with Cloudflare Workers serving the static frontend.

Testing

The backend tests focus on the stuff that's actually easy to get wrong here: does the `seqlock` correctly detect a torn read, does the bar pointer move to the right slot after a close, does the clock always move forward and never jump backward, does a crashed publish clean up after itself and recover. There's also a set of guided end-to-end flows that run in Playwright against the real UI.